Lab Report

Parallel Computing in Python: Estimating Pi with the Monte Carlo Method

1. Goal

To measure how the run time of a program changes when we use more threads.

2. Method

The program (Python, NumPy, Numba) makes 120,000,000 random points in a square with side 1. It counts the points inside a quarter circle. This share is about pi/4, so pi is 4 times the share. The loop uses `prange`, so the threads share the work. I ran it on a MacBook Air with 1, 2, 4 and 8 threads. Each test ran one time.

Speedup: S = T1 / Tn. Efficiency: E = S / n * 100%.

3. Results

Threads	Time (s)	Speedup (x)	Efficiency (%)
1	1.7812	1.00	100.0
2	0.8658	2.06	102.9
4	0.4377	4.07	101.7
8	0.3932	4.53	56.6

Table 1. Measured results

4. Discussion

With 1, 2 and 4 threads the speedup is almost equal to the number of threads. Efficiency a little over 100% is probably a measurement error, because I ran each test only once.

With 8 threads the speedup is only 4.53 and the efficiency is 56.6%. Possible reasons: the processor has fast and slow cores, the MacBook Air has no fan and can slow down when it is hot, and threads need extra time to start and finish.

5. Conclusion

The speedup is almost perfect up to 4 threads. With 8 threads it is much lower, so more threads do not always give a much faster program.